08강

프로그래밍 언어론

변수의 영역

컴퓨터과학과 이관용 교수

학습목차



1 영역의 개요

2 영역 규칙

3 이름 공간

01

영역의 개요

변수의 영역 scope

- 프로그램에서 변수를 사용할 수 있는 범위
  - 변수에 값을 대입하거나 값을 읽어올 수 있는 범위
- 영역의 시작 → 변수가 선언되는 곳

C 언어

오류 발생

```
x12 = 0;
int x12;
x12 = 1 + 5 * 2;
if x12 > 10 then ...
```

변수 x12의 영역

Python

```
x12 = 0
x12 = 1 + 5 * 2
if x12 > 10 then ...
```

오류 없이 동작

변수의 영역과 수명의 관계

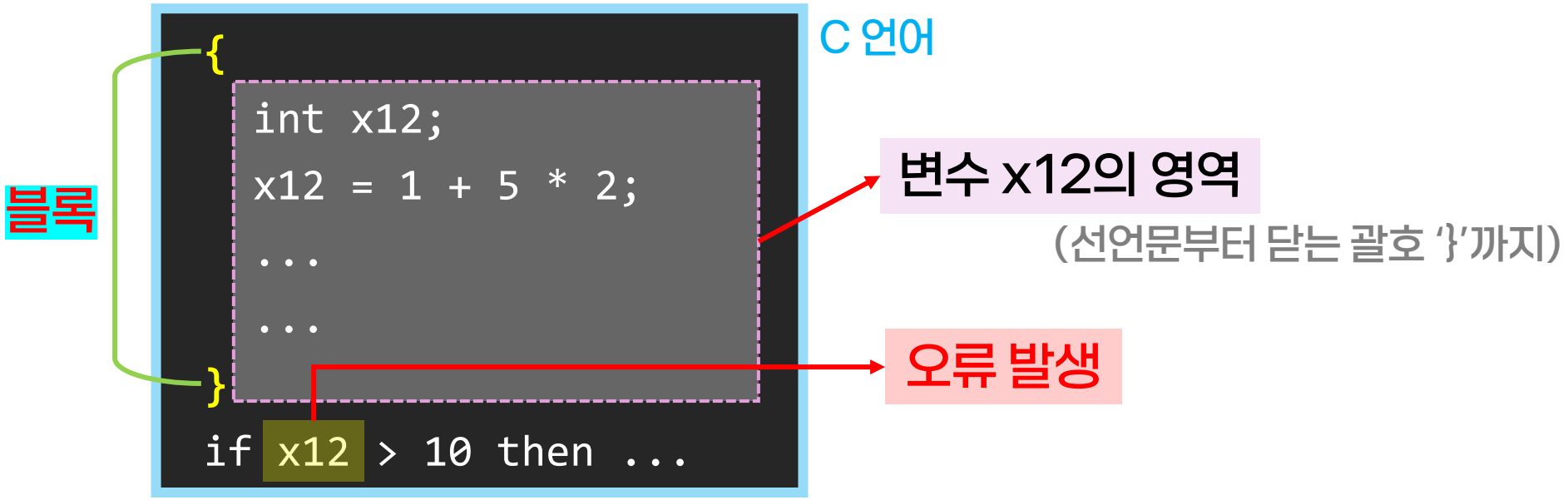
Python

```
x12 = 0
def f():
  x12 = 1 + 5 * 2
if x12 > 10 then ...
```

	x12	x12
영역	프로그램 전체	함수 내부
수명	프로그램 전체	함수가 호출된 시간 동안으로 한정

블록 block

- 프로그램 문장들의 묶음
  - 영역을 구분해 주는 단위로 사용
  - 블록 안에서는 변수 선언이 가능
    - 영역의 시작 → 변수가 선언된 곳. 영역의 끝 → 블록이 끝나는 지점



블록

- 블록의 표현이나 적용 기준은 언어별로 다름
  - Algol 60 → 복합문(`begin ~ end`)
  - C, C++, Java → 복합문(`{ ~ }`), 함수, 클래스
  - Pascal → 주프로그램, 서브프로그램
    - 복합문(`begin ~ end`)은 블록이 아님

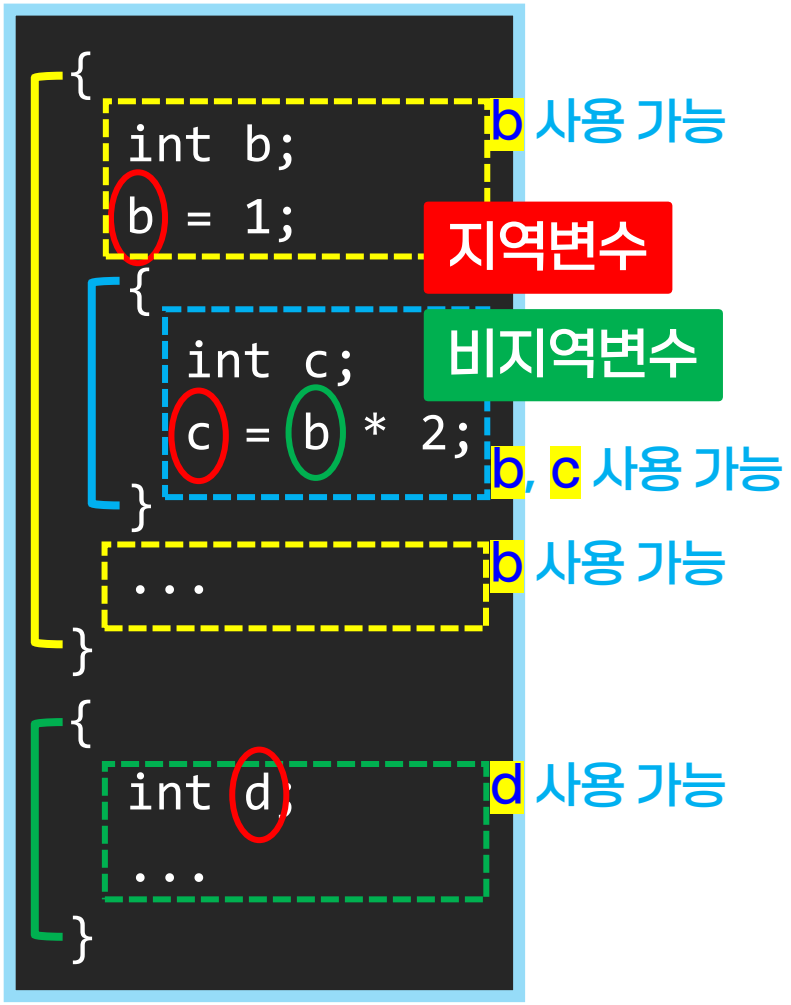
블록과 변수

지역변수 local variable

- 해당 블록 안에서 선언되어 사용되는 변수

비지역변수 non-local variable

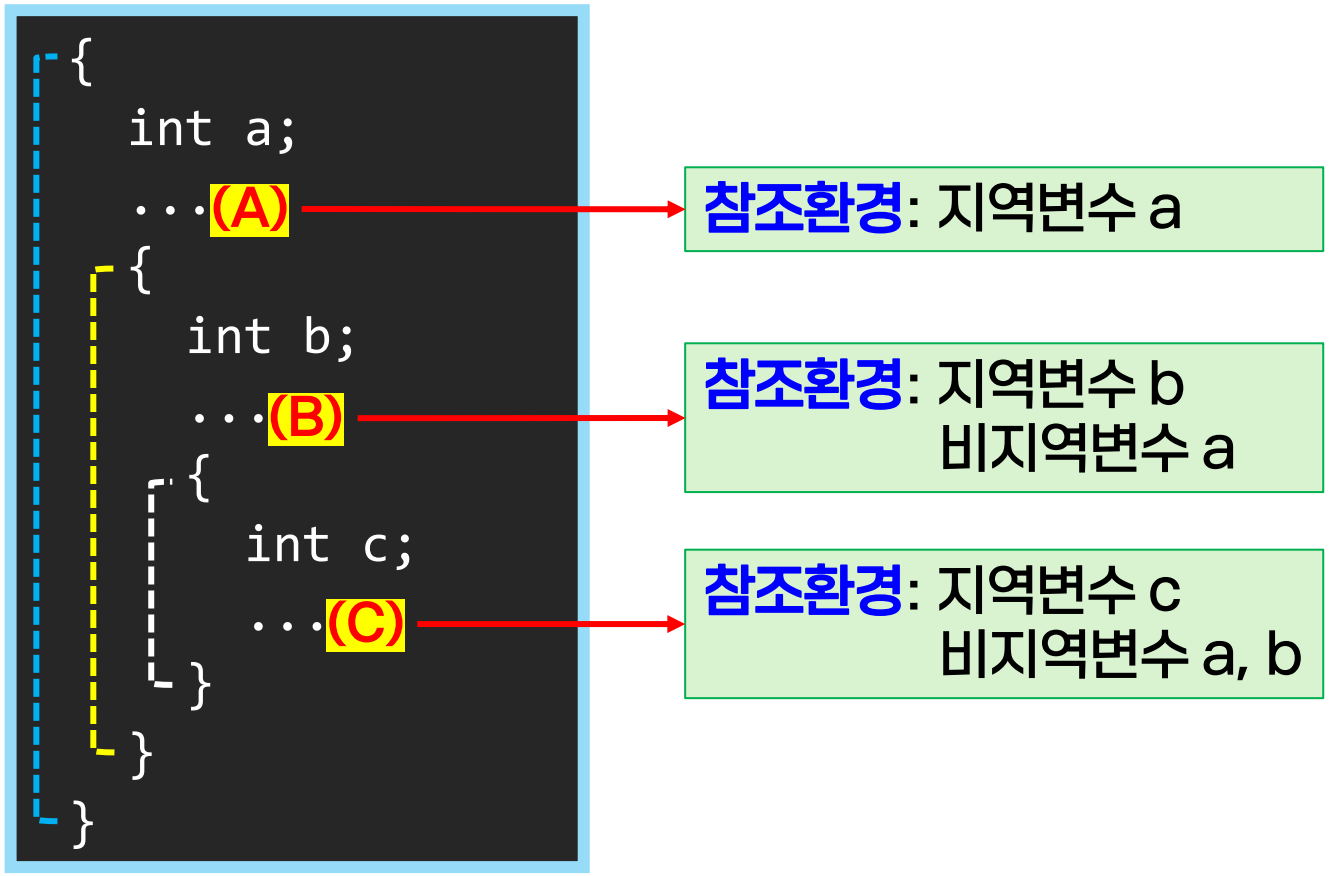
- 프로그램의 특정 지점에서 사용 가능하지만 해당 블록에서 선언되지 않은 모든 변수



- 지역변수와 비지역변수는 상대적인 개념
  - 지역변수는 자신이 포함하고 있는 블록 내에서는 비지역변수로 사용 가능

참조 환경 referencing environment

- 프로그램의 한 위치에서 사용할 수 있는 모든 식별자의 모음
  - 해당 위치의 모든 지역변수와 모든 비지역변수를 포함



02

영역 규칙

영역 규칙 scope rule

- 변수의 참조 위치를 결정하는 방법
  - 비지역변수의 위치를 어떻게 결정할 것인지를 주로 취급
  - 종류 → 정적 영역 규칙 static scope rule, 동적 영역 규칙 dynamic scope rule
- 자유변수 free variable
  - 현재 블록에서 선언되지 않고 사용하려는 변수
  - 영역 규칙에 따라 참조 위치를 결정
    - 참조 위치를 찾은 경우(선언된 경우) → 비지역변수
    - 참조 위치를 찾지 못한 경우 → 오류

자유변수의 예

```
program Main;
  A: Integer;
  procedure Sub1;
  begin {sub1}
    ...A...
  end;
  procedure Sub2;
    A: Integer;
  begin {Sub2}
    sub1
  end;
begin {Main}
  ...
  Sub2
end.
```

Pascal

자유변수 A

비지역변수? or 오류?

- Main에서 선언된 변수 A?
- Sub2에서 선언된 변수 A?

정적 영역 규칙

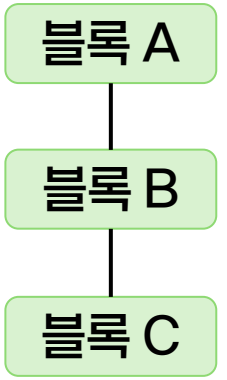
- 블록들의 정적 내포 관계를 이용하여 변수의 참조 위치를 결정
  - 정적 내포 관계 → 블록들의 문맥적인 포함 관계
    - "정적" → "프로그램 수행 없이 내포 관계의 판단이 가능하다"
  - "어휘적 영역 규칙 lexical scope rule"

- 용어
  - 정적 조상 static ancestors
    → 현 블록을 문맥적으로 포함하는 모든 블록
  - 정적 부모 static parent
    → 현 블록에 가장 가까운 정적 조상

```
{ // 블록 A
  int a;
  ...
  { // 블록 B
    int b;
    ...
    { // 블록 C
      int c;
      ...
    }
  }
}
```

A
B
C

블록의 정적 내포 관계의 트리 표현



정적 영역 규칙의 적용 방법

정적 영역 규칙의 적용

1. 사용하려는 변수의 이름에 대한 선언이 현재 블록 안에 존재
   → 지역변수

2. 자유변수라면 현재 블록의 정적 부모에 대해
   ① 자유변수 이름에 대한 선언이 존재 → 비지역변수
   ② 선언이 없으면 그 블록의 정적 부모에 대해 반복

3. 최외곽 영역까지 선언을 찾지 못함 → 오류

정적 영역 규칙의 적용 예_1

Pascal

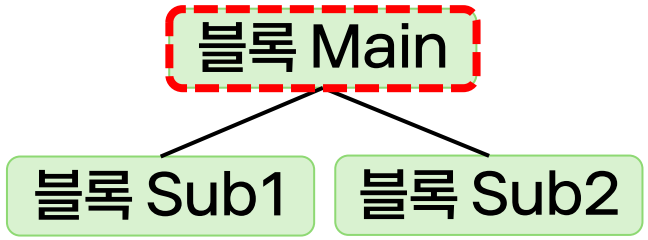
```
program Main;
  A: Integer;
  procedure Sub1;
  begin {sub1}
    ...A...
  end;
  procedure Sub2;
    A: Integer;
  begin {Sub2}
    sub1
  end;
begin {Main}
  ...
  Sub2
end.
```

자유변수 A

비지역변수

Main에서 선언된 변수 A

블록의 정적 내포 관계의 트리 표현



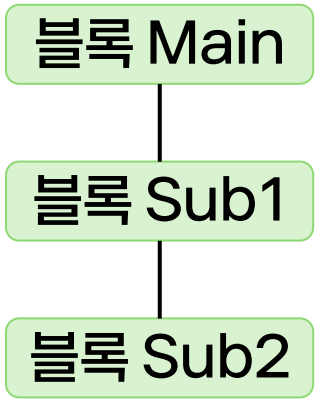
정적 영역 규칙의 적용 예_2

Pascal

```
program Main;
  A: Integer;
  procedure Sub1;
    A: Integer;
    procedure Sub2;
    begin {Sub2}
      ...A...
    end;
  begin {Sub1}
    Sub2
  end;
begin {Main}
  ...
  Sub1
end.
```

자유변수 A

블록의 정적 내포 관계의 트리 표현



`A: Integer;` 이 없는 경우라면?

`A: Integer;` 도 없는 경우라면?

영역 구멍 scope hole

- 비지역변수가 같은 이름의 지역변수 때문에 보이지 않는 영역

Sub2의 변수 A의 영역
→ Main/Sub1의 변수 A의 영역 구멍

Sub1의 변수 A의 영역
→ Main의 변수 A의 영역 구멍

Main의 변수 A의 영역

Pascal

```
program Main;
  A: Integer;          A=10
  procedure Sub1;
    A: Integer;        A=20
    procedure Sub2;
      A: Integer;      A=30
    begin {Sub2}
      ...A.            출력 A?
    end;
  begin {Sub1}
    Sub2;              출력 A?
  end;
begin {Main}
  ...                  출력 A?
  Sub1
end.
```

동적 영역 규칙

- 블록들의 동적 내포 관계를 이용하여 변수의 참조 위치를 결정
  - 동적 내포 관계 → 서브프로그램의 호출 관계
  - 서브프로그램의 호출에 따른 포함 관계는 프로그램 수행 시점에서만 판단 가능

동적 영역 규칙의 적용 방법

동적 영역 규칙의 적용

1. 사용하려는 변수의 이름에 대한 선언이 현재 블록 안에 존재
   → 지역변수

2. 자유변수라면 현재 블록을 호출한 블록에 대해
   ① 자유변수 이름에 대한 선언이 존재 → 비지역변수
   ② 선언이 없으면 그 블록을 호출한 블록에 대해 반복

3. 최초 호출자 영역까지 선언을 찾지 못함 → 오류

동적 영역 규칙의 적용 예_1

```
program Main;
  A: Integer;
  procedure Sub1;
  begin {sub1}
    ...A...
  end;
  procedure Sub2;
    A: Integer;
  begin {Sub2}
    sub1
  end;
begin {Main}
  ...
  Sub2
end.
```

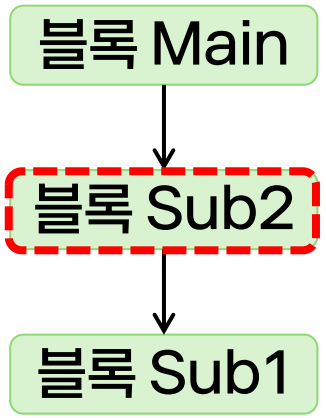
Pascal

자유변수 A

비지역변수

Sub2에서 선언된 변수 A

서브프로그램의 호출 관계



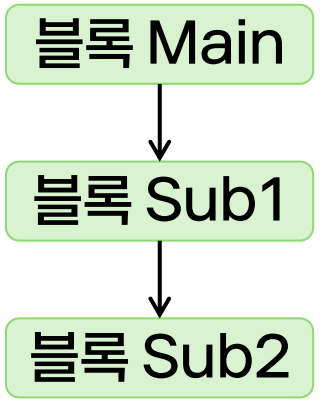
동적 영역 규칙의 적용 예_2

Pascal

```
program Main;
  A: Integer;
  procedure Sub1;
    A: Integer;
    procedure Sub2;
    begin {Sub2}
      ...A...
    end;
  begin {Sub1}
    Sub2
  end;
begin {Main}
  ...
  Sub1
end.
```

자유변수 A

서브프로그램의 호출 관계



`A: Integer;` 이 없는 경우라면?

`A: Integer;` 도 없는 경우라면?

영역 규칙의 비교

	정적 영역 규칙	동적 영역 규칙
결정 시점	컴파일 시점	프로그램 수행 시점
특징	정적 타입 검사 가능 빠른 수행 속도	정적 타입 검사 불가능 느린 수행 속도
적용 언어	Algol 60, Pascal, C, C++, Java, Python 등 대부분	LISP 등 인터프리터 방식의 구현 언어

03

이름공간

전역 변수 global variable

- 어떤 블록에도 포함되지 않는 곳에서 선언된 변수
  - 영역 → 프로그램 전체
  - 모든 블록에서 비지역변수로 취급

```
int g;
{ // 블록 A
  int a;
  ...
  { // 블록 B
    int b;
    ...
  }
}
```

전역변수 g

전역변수명과 동일한 이름의 지역변수를 선언하는 경우

↓

영역 구멍 발생

영역 구멍 없는 전역변수의 사용 방법

- 영역 연산자 scope operator, scope resolution operator 이용
  - C++의 영역 연산자 → '::변수명'

```
#include <iostream>
int i=0;          전역변수 i

int main()
{
   int i=5;       지역변수 i
   i++;
   ::i++;
   cout << "local i=" << i << endl;
   cout << "nonlocal i=" << ::i << endl;
   return 0;
}
```

```
local i=6
nonlocal i=1
```

이름 공간 namespace

- 관련성이 높은 변수와 함수를 하나의 묶음으로 관리하는 영역
  - 영역 자체의 이름을 가짐

```
namespace S1 {
  int k=0;
  void Output() {
    cout << "k=" << k << endl;
  }
}
```

이름 공간

- 이름 공간 내의 변수/함수를 이름 공간 밖에서 사용하는 방법

① 이름 공간의 이름과 영역 연산자를 변수명/함수명 앞에 붙여서 사용하는 방법

② 예약어 using을 사용하여 이름 공간 자체의 영역을 확장하는 방법

C++

```
#include <iostream>
namespace S1 {
 int k=0;
 void Output() {
  cout << "k=" << k << endl;
 }
}

int main(void) {
 S1::k++;
 S1::Output();          ..... k=1

 using namespace S1;
 k++;
 Output();              ..... k=2
 return 0;
}
```

이름 공간

- 이름 공간과 영역 구멍
  - 이름 공간의 변수와 지역 변수가 같은 이름을 갖는 경우

  → 영역 연산자를 이용해서 해결

C++

```
#include <iostream>
namespace S1 {
 int k=0;
 void Output() {
  cout << "k=" << k << endl;
 }
}

int main(void) {
 int k=5;

 using namespace S1;

 k++;                    // 지역변수 k=6
 S1::k++;
 Output();  ..... k=1
 return 0;
}
```

정리하기

영역의 개요

- 영역 → 프로그램에서 변수를 사용할 수 있는 범위
- 영역 vs 수명, 블록(프로그램 문장의 묶음-영역 구분 단위), 지역변수 vs 비지역변수
- 참조환경 → 프로그램의 한 위치에서 사용할 수 있는 모든 식별자/변수의 모음

영역 규칙

- 영역 규칙(변수의 참조 위치 결정 방법), 자유 변수, 영역 구멍
- 정적 영역 규칙 → 문맥적 포함 관계, 컴파일 시점, 정적 타임 검사 가능, 빠른 수행 속도
- 동적 영역 규칙 → 서브프로그램 호출 관계, 수행 시점, 정적 타임 검사 불가, 느린 수행 속도

이름 공간

- 전역 변수 → 어떤 블록에도 포함되지 않는 곳에서 선언된 변수, 모든 블록에서 비지역변수
- 이름 공간 → 관련성이 높은 변수와 함수를 하나의 묶음으로 관리하는 영역
- 영역 구멍 문제는 영역 연산자(::) 또는 예약어 using을 이용해서 해결

09강

다음시간안내 ▶▶▶

타입